RESEARCH PRESENTATION

From Generating SQL to Trusting SQL

Toward Reliable LLM-based Text-to-SQL Systems

Ke Lu

COMS 6156 | Columbia University

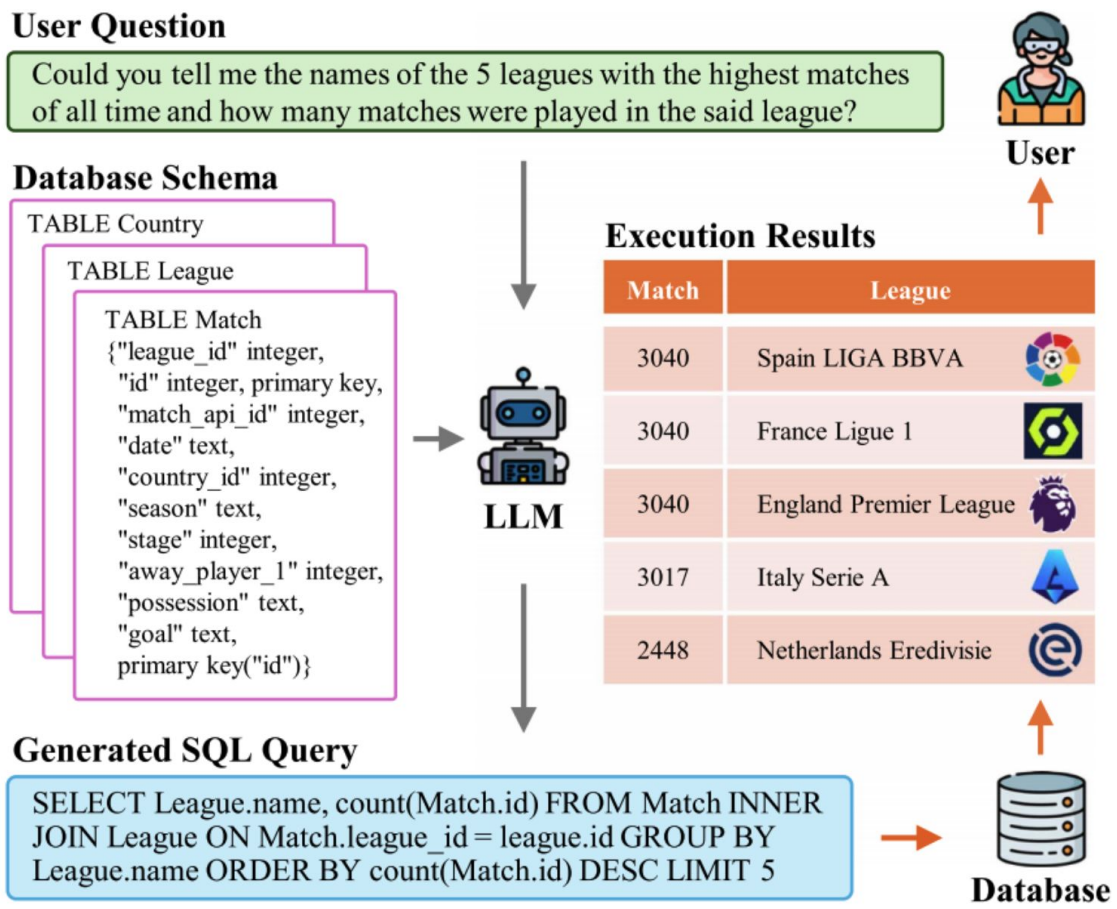
Problem Overview

- Translates **Natural Language → SQL queries**
- Eliminates the need for deep SQL expertise
- Powered entirely by **Large Language Models (LLMs)**

EXAMPLE TRANSLATION

User: "Show total sales in 2023"

```
SELECT SUM(sales)
FROM orders
WHERE year = 2023
```



Source: Next-Generation Database Interfaces: A Survey of LLM-Based Text-to-SQL

Why Text-to-SQL Matters

Natural Language Interfaces

01

Bridging the gap between human intent and complex database structures seamlessly.

Accessibility

02

Enabling non-technical users to access and analyze data without needing to write SQL.

BI Automation

03

Streamlining Business Intelligence processes for faster decision-making.

Conversational AI

04

Serving as the core engine behind intelligent data-driven virtual assistants.

Real-world Applications → Enterprise Data Systems Analytics Platforms AI Assistants

Progress with LLMs

Explosive Growth Since 2023

LLMs have significantly improved SOTA, breaking previous performance barriers in code generation through advanced **Prompt Engineering** & **Reasoning Strategies**.

Key Benchmarks

- Spider Dataset
- WikiSQL Dataset

Notable Models

- SQLifyMe
- PSM-SQL
- ToolSQL

Primary Evaluation Metric

Execution Accuracy (EX)

Executable ≠ Correct

The Illusion

LLM-generated SQL often passes syntax checks and **runs successfully** without throwing any database errors.

Result: Low trust in the generated output due to "silent failures."

The Reality: Common Errors

Queries produce incorrect semantic results due to:

- Wrong table selection
- Incorrect join paths between tables
- Mismatched or hallucinated condition values
- Missing implicit domain knowledge

Improving Text-to-SQL Reliability

Three Major Research Directions to Bridge the Reliability Gap

Schema Awareness

Understanding DB structures, linking, and grounding before generation.

Knowledge & Reasoning

Generating intermediate logic steps to align intent with data.

Guardrails & Verification

Validating outputs, checking constraints, and self-correcting errors.

I. Schema Awareness

- Understand database structure first
- **Methods:** Schema linking, table retrieval
- Column filtering & semantic matching
- **Goal:** Reduce structural errors
- Improve table/column selection

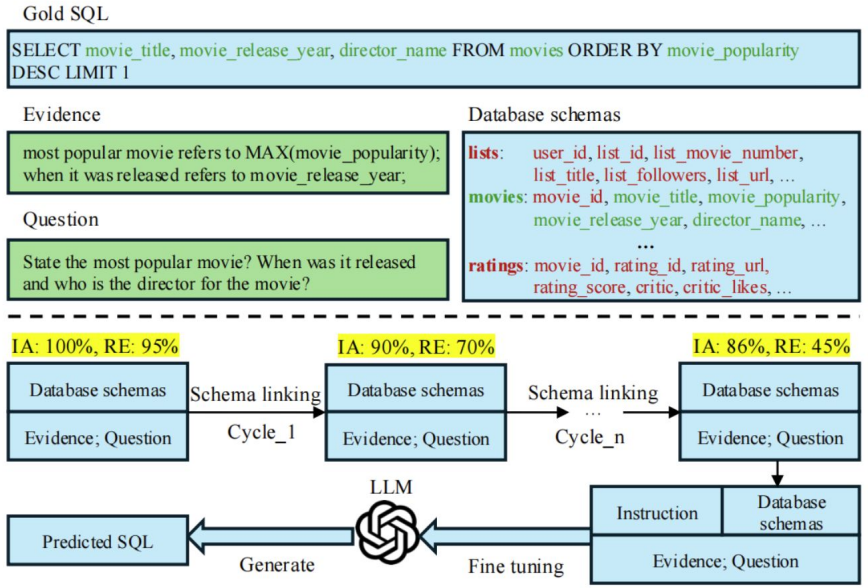


Figure 1: Illustration of an example of Text-to-SQL solved by PSM-SQL. IA and RE denote the Including Accuracy and Redundancy, respectively.

Source: PSM-SQL: Progressive Schema Learning with Multi-granularity Semantics for Text-to-SQL

II. Knowledge & Reasoning

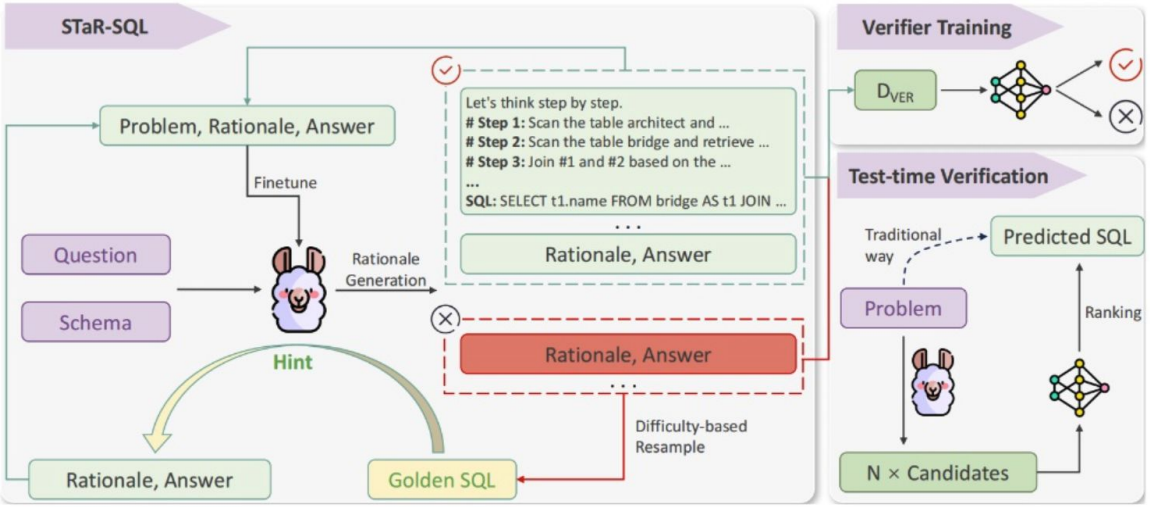


Figure 2: An overview of the STaR-SQL framework. It consists of three main steps: step-by-step rationale generation for self-improvement, verifier training, and test-time verification. We transform text-to-SQL into a reasoning task and further explore scaling up test-time computation by incorporating a verifier and employing best-of-N sampling.

- Generate reasoning steps before SQL
- **Chain-of-Thought (CoT)** reasoning
- STaR-SQL self-training
- Knowledge injection
- Better query understanding
- Interpretable logic

III. Guardrails & Verification

- Verify SQL after generation
- **Typical Checks:**
  - Column existence
  - Join path validation
  - Execution-based checking
- Goal: Detect and correct errors automatically

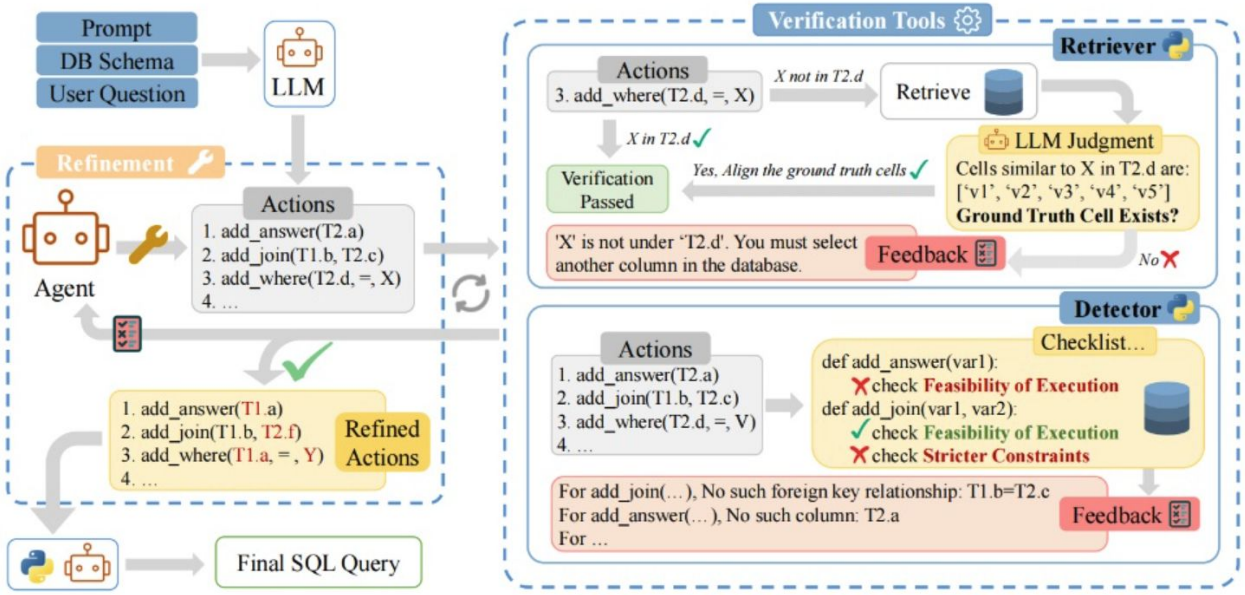


Figure 2: The overall structure of `ToolSQL`, consisting of an LLM-based agent for SQL refinement (left) and two tools for error verification (right). The LLM-based agent performs actions that represent the SQL query and continuously refines these actions based on feedback from the tools. The tools target different problems in SQL queries: (i) the *Database Retriever* detects condition mismatches, and (ii) the *Error Detector* diagnoses mismatches of stricter constraints and execution errors.

Source: ToolSQL: A Tool-Assisted Agent for SQL Verification and Refinement

Research Gap

Fragmented Optimization

Methods are often optimized independently. Schema, reasoning, and verification are rarely integrated into a single pipeline.

Benchmark Limitations

Current evaluation datasets (like Spider) are overwhelmingly "clean".

> They lack the complexity and messiness found in actual enterprise environments.

Real-world Challenges

- Highly ambiguous human language
- Incomplete query conditions & assumptions
- Missing domain metadata and context

My Research Direction

Reliability-oriented Text-to-SQL System

Schema Awareness + Knowledge / Reasoning + Guardrails

Executable SQL

Reliable Query Results

Interpretable Reasoning

Self-correcting Pipelines

Thank you for your time

Discussion Questions